

HW 1 - Written portion

Problem 1

a)

counting iterations: i changes by being squared each time: $2, 2^2, 2^4, 2^8$ (exponent doubles)

$\therefore$ after t iterations $\rightarrow i = 2^{2^t}$

loop continues while $i < n$ so stops when $2^{2^t} \geq n$.

$\hookrightarrow 2^{2^t} \geq \log n \rightarrow t \geq \log(\log n)$ so...

$$T(n) = \Theta(\log \log n)$$

b)

outer loop runs n times. condition check = $\Theta(n)$ work but inner loop only runs when i is divisible by $\sqrt{n}$

so let $s = \sqrt{n}$ $q = \frac{n}{s}$

the condition is true at $i = s, 2s, \dots, qs$

both s and q are $\Theta(\sqrt{n})$ so there are $\Theta(\sqrt{n})$ successful checks. For each check (successful) the inner loop performs (i^2) iterations.

$$\sum_{j=1}^q (js)^3 = s^3 \sum_{j=1}^q j^3$$

$$\rightarrow \left(\frac{q(q+1)}{2} \right)^2 = \Theta(q^4) \rightarrow \Theta(s^3 q^4) = \Theta(n^{7/2})$$

$$\Theta((\sqrt{n}^3)(\sqrt{n}^4)) = \Theta(n^{7/2})$$

$$T(n) = \Theta(n) + \Theta(n^{7/2}) = \Theta(n^{7/2})$$

c)

i loop runs n times, and k loop runs n times per i. so $ARR[i] == i$ is checked n^2 times.

$\hookrightarrow \Theta(n^2)$

the condition can succeed at most n times.

$\hookrightarrow$ when it succeeds, m starts at 1 and doubles each iteration. so after t iterations $m = 2^t$

$\hookrightarrow$ one complete run = $\Theta(\log n)$

$\hookrightarrow O(n) * \Theta(\log n) = O(n \log n)$

$T(n) = \Theta(n^2) + O(n \log n) \rightarrow T(n) = \Theta(n^2)$

d)

initialization takes constant time.

outer loop executes n times. $\Theta(n)$

resizing: one resize = $\Theta(\text{size})$

L = last resizing so:

$L, \frac{2}{3}L, \left(\frac{2}{3}\right)^2 L \dots$ (bounded by $3L$)

total copies $\leq L \sum_{i=0}^{\infty} \left(\frac{5}{7}\right)^i = \frac{7}{2}L = O(L)$

Last resize is when $\text{size} = i < n$ so $L < n$

$\hookrightarrow O(n)$

$T(n) = \Theta(n) + O(n) = \Theta(n)$

Problem 2:

a) when both inputs aren't null function keeps first node of $in1$ and sets next pointer to result of

$llrec(in2, in1 \rightarrow next)$

let px = pointer to the node containing x

recursive calls:

$llrec(p1, p5)$

$llrec(p5, p2)$

$llrec(p2, p6)$

$llrec(p6, p3)$

$llrec(p3, nullptr)$

↳ returning head is $p1$

b)

call is

$llrec(nullptr, p2)$

first condition, $in1 == nullptr$ is true so

return $in2$;

returns $p2$ without making recursive calls or changing any pointers. so returned list contains only 2.